CS528 Homework 6

Database Normalization and Predictive Models

Yide Fang | Boston University | November 3, 2026

Code: https://github.com/ThomasFYD/CS528-HWs
Directory: **CS528-HW6**

1. Design, data and measured results

I used the complete formal HW5 experiment: 100,000 requests, 76,603 distinct client IPs and 197 country labels. The source run is **load-20261021T075735Z-5b3f60c2**. Other HW5 tests remain in the original database.

The implementation creates four normalized base tables in a separate **hw6** database on the preserved Cloud SQL instance. It verifies that joining the tables reconstructs every original field before committing the data migration.

Model	Held-out accuracy	Result
Client IP to country	100.00% (19,975/19,975)	Meets the 99% requirement
Available fields to income	34.27% (6,854/20,000)	Below the 35% aim; limitation reported

Country prediction uses a learned IP-prefix classifier. Income prediction compares three candidates on a separate validation set and evaluates the selected model on a fixed test set. The seed is 528. Section 5 explains both models, the partitions and the income result.

Execution and retained evidence

The models were run on a newly created VM against Cloud SQL. Source and reconstructed record hashes match. Output downloads passed SHA-256 checks. Cleanup verified that SQL was stopped and the temporary VM and network were removed.

Bucket output: gs://cs528-hw2-yf-2026/cs528-hw6/runs/20261103-015057-b504/

Source, lifecycle scripts and measured-run evidence are provided in CS528-HW6. The report/evidence directory contains the migration checks, model metrics, output hashes and cleanup record. Full prediction files and split membership are retained in the bucket.

Scope of the result

The country result reflects instructor-generated IP patterns. The income result is below the assignment aim and is reported without changing the test partition or seed. The original HW5 tables and the normalized HW6 database are retained for inspection; the temporary compute resources have been removed.

2. Functional dependencies and 3NF

The original request table has the transitive dependency **request_id -> client_ip -> country -> is_banned**. It also stores **client_time -> time_of_day**. These determinants are not request-table candidate keys, so the original table is not in 3NF.

Base table / key	Stored attributes and dependency
countries / country	is_banned; country determines the fixed homework policy flag.
clients / client_ip	country (FK); a simulated client IP determines exactly one country.
requests / request_id; request_time is also UNIQUE	request_time, run_id, method, requested_file, client_ip (FK), peer_ip, gender, age, income, client_time, http_status. Each value describes a request.
request_timings / request_id (PK and FK)	extract_ns, storage_ns, response_ns, database_ns, total_ns, storage_attempted, response_sent. One row per request.

All stored values are atomic for their intended domain. Each primary key is a single attribute, so there are no partial key dependencies. Under the assignment assumptions, every non-key attribute in each base table depends on a candidate key, with the identified transitive dependencies removed. Age and income ranges are categorical values; IP is not assumed to determine a client's demographic attributes.

Derived values and failure records

The request_details view joins requests, clients and countries, and derives time_of_day from client_time. The failed_requests view derives non-200 request rows. The old failure table has its own valid key, but copying file/status values into another stored table is unnecessary here. Source error rows are checked for exact agreement with the view.

Data-transfer safeguards

Before migration, the program rejects missing metadata, conflicting IP-country pairs, inconsistent country-policy flags, duplicate keys, mismatched derived times, inconsistent failures and missing timing records. The formal run contains zero IP-country conflicts and zero missing required fields.

Data INSERTs share one transaction. SHA-256 fingerprints cover every field of reconstructed requests, failures and timings, with canonical ordering and MySQL boolean/date normalization. A mismatch rolls back the new INSERTs. Existing rows are retained and checked on rerun. MySQL DDL runs separately because it commits implicitly.

3. MySQL schema

The orchestration script first creates the hw6 database. The following DDL defines the normalized base tables.

```
-- MySQL 8.4. The runner creates the separate hw6 database first.
-- Source hw5 tables remain unchanged. Only the complete formal HW5 run is migrated.
CREATE TABLE IF NOT EXISTS hw6.countries (
    country VARCHAR(128) NOT NULL PRIMARY KEY,
    is_banned BOOLEAN NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin;

CREATE TABLE IF NOT EXISTS hw6.clients (
    client_ip VARCHAR(45) NOT NULL PRIMARY KEY,
    country VARCHAR(128) NOT NULL,
    FOREIGN KEY (country) REFERENCES hw6.countries(country)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin;

CREATE TABLE IF NOT EXISTS hw6.requests (
    request_id VARCHAR(36) NOT NULL PRIMARY KEY,
    request_time DATETIME(6) NOT NULL UNIQUE,
    run_id VARCHAR(64),
    method VARCHAR(32) NOT NULL,
    requested_file VARCHAR(1024) NOT NULL,
    client_ip VARCHAR(45) NOT NULL,
    peer_ip VARCHAR(45),
    gender VARCHAR(32),
    age VARCHAR(32),
    income VARCHAR(32),
    client_time VARCHAR(64),
    http_status INTEGER NOT NULL,
    FOREIGN KEY (client_ip) REFERENCES hw6.clients(client_ip),
    INDEX (run_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin;

CREATE TABLE IF NOT EXISTS hw6.request_timings (
    request_id VARCHAR(36) NOT NULL PRIMARY KEY,
    extract_ns BIGINT NOT NULL,
    storage_ns BIGINT NOT NULL,
    response_ns BIGINT NOT NULL,
    database_ns BIGINT NOT NULL,
    total_ns BIGINT NOT NULL,
    storage_attempted BOOLEAN NOT NULL,
    response_sent BOOLEAN NOT NULL,
    FOREIGN KEY (request_id) REFERENCES hw6.requests(request_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin;
```

4. Reconstruction and migration queries

The runner binds :run_id to the formal HW5 run. MIN selects the single value only after functional-dependency validation; it does not resolve conflicts.

```
CREATE OR REPLACE VIEW hw6.request_details AS
SELECT r.request_id, r.request_time, r.run_id, r.method, r.requested_file,
       c.country, r.client_ip, r.peer_ip, r.gender, r.age, r.income,
       co.is_banned, r.client_time,
       TIME_FORMAT(STR_TO_DATE(r.client_time, '%Y-%m-%d %H:%i:%s'), '%H:%i:%s') AS time_of_day,
       r.http_status
FROM hw6.requests r JOIN hw6.clients c USING (client_ip)
JOIN hw6.countries co USING (country);

CREATE OR REPLACE VIEW hw6.failed_requests AS
SELECT request_time, requested_file, http_status AS error_code
FROM hw6.requests WHERE http_status <> 200;

INSERT INTO hw6.countries (country, is_banned)
SELECT r.country, MIN(r.is_banned) FROM hw5.requests r
LEFT JOIN hw6.countries d ON d.country = r.country
WHERE r.run_id = :run_id AND d.country IS NULL
GROUP BY r.country;

INSERT INTO hw6.clients (client_ip, country)
SELECT r.client_ip, MIN(r.country) FROM hw5.requests r
LEFT JOIN hw6.clients d ON d.client_ip = r.client_ip
WHERE r.run_id = :run_id AND d.client_ip IS NULL
GROUP BY r.client_ip;

INSERT INTO hw6.requests
(request_id, request_time, run_id, method, requested_file, client_ip,
 peer_ip, gender, age, income, client_time, http_status)
SELECT r.request_id, r.request_time, r.run_id, r.method, r.requested_file,
       r.client_ip, r.peer_ip, r.gender, r.age, r.income, r.client_time, r.http_status
FROM hw5.requests r LEFT JOIN hw6.requests d ON d.request_id = r.request_id
WHERE r.run_id = :run_id AND d.request_id IS NULL;

INSERT INTO hw6.request_timings
(request_id, extract_ns, storage_ns, response_ns, database_ns, total_ns,
 storage_attempted, response_sent)
SELECT t.request_id, t.extract_ns, t.storage_ns, t.response_ns, t.database_ns,
       t.total_ns, t.storage_attempted, t.response_sent
FROM hw5.request_timings t JOIN hw5.requests r USING (request_id)
LEFT JOIN hw6.request_timings d ON d.request_id = t.request_id
WHERE r.run_id = :run_id AND d.request_id IS NULL;
```

5. Models and held-out evaluation

Model 1: learned IP-prefix classifier

The classifier learns the most frequent country for each /24 IPv4 prefix from training rows only. For an unseen prefix, it backs off to /16, /8, then the training majority. This is a learned categorical lookup model. It does not consult a full-data IP-country table when predicting test labels.

With seed 528, 80% of sorted unique IPs are assigned to training and the remainder to test. This produces 80,025 training requests and 19,975 test requests, with **zero shared IPs**. Test accuracy is **100.00%**. The result reflects instructor-generated IP patterns and is not evidence of real-world geolocation performance.

Model 2: income-range classification

Sort requests by ID and shuffle once using seed 528. Use 60,000 rows for candidate fitting, 20,000 for validation, and reserve 20,000 for final testing. Compare three fixed candidates. Select the highest validation accuracy, refit that model on the first 80,000 rows, and evaluate the fixed test set. No test labels, income-derived inputs or request IDs enter feature learning.

Candidate	Validation accuracy
age_mode	33.375%
random_forest	33.655%
hist_gradient_boosting	33.205%

Random forest uses age, gender, country and simulated hour, one-hot encoding fitted only on training data, 120 trees, maximum depth 18, minimum 20 examples per leaf, max_features=0.8 and random seed 528. The alternatives are a training age-group mode and histogram gradient boosting with 150 iterations, 15 leaves, learning rate 0.08 and L2 penalty 10.

The selected model is **random_forest**. Its test accuracy is **34.27%**, versus a training-majority baseline of **12.675%**. Exact category matching is used. The approximate 95% Wilson interval is 33.62% to 34.93%; repeated demographic patterns mean the usual independent-trial interval should be interpreted cautiously.

Income target and limitations

The measured result is 0.73 percentage points below the 35% aim. The assignment warns that generated data and the random seed can affect fitting. The seed and test partition were kept fixed. The compared models do not establish a theoretical accuracy ceiling.

Additional local optimization

A follow-up compared 27 configurations on the original training and validation partitions, including additional field combinations, Extra Trees and CatBoost. The best validation score was **34.590%** (CatBoost, depth 4, 99 trees). Its settings were frozen, then refitted on 80,000 rows. One follow-up evaluation on the original test set scored **33.295% (6,659/20,000)**. This local result did not improve the published cloud model; the original model and bucket outputs are retained.

The test set had already been used for the original model; it is not a new independent holdout. Only training/validation data guided this search. Each training age group had three nearly equally frequent income labels; added fields did not show reliable improvement. All 27 scores, settings, diagnostics and final-test checksums are in report/evidence/income-optimization.json.

6. Configuration, execution and cleanup

Required local tools: Google Cloud CLI and Python 3.11 or 3.12. Authenticate with an authorized project identity; the owner uses fangyd@bu.edu. Every cloud command pins project dazzling-seat-508219-j3.

```
gcloud auth login fangyd@bu.edu
cd CS528-HW6
python3 infra.py plan
python3 infra.py preflight
bash run.sh
```

Setting	Value
Existing SQL	cs528-hw5-mysql; MySQL 8.4; source hw5; target hw6
VM	Temporary e2-standard-2; Debian 12; 10 GB standard boot disk
Location	us-central1-a; dedicated temporary VPC and subnet
Password	Secret Manager: cs528-hw5-db-password; no committed key
Output bucket	gs://cs528-hw2-yf-2026
Identity	cs528-hw6-worker; Cloud SQL Client, one secret, scoped storage access

The script checks that SQL is stopped, prepares the identity, uploads a source archive, starts SQL and creates the VM. A startup script downloads the archive and validates its SHA-256, installs pinned dependencies, migrates SQL data and runs the models. No interactive SSH configuration is required. Predictions and evidence are uploaded under a unique run prefix; done.json is written last.

The controller verifies downloaded output hashes, then stops SQL and deletes the VM with its boot disk, subnet and network. It prints summary.txt and both complete prediction CSVs from the downloaded bucket objects. SQL data, outputs, source archives and the reusable identity are retained. Stopped SQL still allocates storage.

Failure handling

Exceptions, Ctrl-C and SIGTERM attempt cleanup. VM deletion is still attempted if stopping SQL fails. Network loss, forced process termination or expired authentication can prevent cleanup. Use the recorded run state to retry:

```
python3 infra.py cleanup --state results/<run-id>/state.json
```

If uploads completed, use the resume action with the same --state path to recover verified outputs, cleanup and terminal printing without starting resources. The measured run required this recovery after a client-side timeout during VM deletion. The original error record is retained. Final cleanup.json must show verified=true.

7. Output examples and evidence

Each prediction file contains the request ID, actual label, predicted label and correctness. Country output also identifies the learned prefix length; income output includes the four candidate input fields.

Country prediction examples

Client IP	Actual country	Prediction	Correct
50.107.250.138	Liechtenstein	Liechtenstein	True
26.142.191.137	Kenya	Kenya	True
132.7.6.94	Morocco	Morocco	True
27.51.95.209	Togo	Togo	True
208.49.233.44	Croatia	Croatia	True

Income prediction examples

Age	Actual income	Prediction	Correct
46-55	40k-60k	40k-60k	True
0-16	250k+	0-10k	False
0-16	250k+	0-10k	False
76+	0-10k	250k+	False
36-45	20k-40k	20k-40k	True

Verification record

Fifteen local tests passed: dependency conflicts, derived-value consistency, failed/timing record integrity, canonical hashes, repeatable SQL transfer, held-out split isolation, target-free features, prefix backoff, cleanup continuation, leftover disk detection, recovery failure handling and ownership checks. Python and shell syntax checks passed.

Measured dependency versions: scikit-learn 1.7.2, numpy 2.2.6, pandas 2.3.3, scipy 1.15.3, SQLAlchemy 2.1.5, PyMySQL 1.1.3

Full per-class reports, confusion matrices, partition membership and hashes are saved in metrics.json, splits.csv and migration.json. Cloud cleanup has been verified.

Sources

1. CS528-HW_6.pdf, assignment instructions supplied by the instructor.
2. Cloud Computing (lecture 13) -- Database optimization.pdf, normal forms, slides 5, 7, 9 and 11.
3. HW5 source schema and formal-run backup in CS528-HW5.
4. scikit-learn ensemble documentation.
5. Google Compute Engine Linux startup scripts.